Computerized Medical Imaging and Graphics 36 (2012) 366–374

Contents lists available at SciVerse ScienceDirect

Computerized Medical Imaging and Graphics

journal homepage: www.elsevier.com/locate/compmedimag

Interactive GPU-based maximum intensity projection of large medical data sets using visibility culling based on the initial occluder and the visible block classification

Heewon Kye[a], Bong-Soo Sohn[b], Jeongjin Lee[c,*]

[a] Department of Information Systems Engineering, Hansung University, 389 Samseon-dong 2-ga, Seongbuk-gu 136-792, Seoul, Republic of Korea
[b] School of Computer Science and Engineering, Chung-Ang University, 221 Heukseok-dong, Dongjak-gu, Seoul, Republic of Korea
[c] Department of Digital Media, the Catholic University of Korea, 43-1 Yeokgok 2-dong, Wonmi-gu, Bucheon-si, Gyeonggi-do 420-743, Republic of Korea

ARTICLE INFO

Article history:
Received 9 August 2011
Received in revised form 3 February 2012
Accepted 9 April 2012

Keywords:
Maximum intensity projection (MIP)
Graphics processing unit (GPU)
Visibility culling
Initial occluder
Block classification
Large volume data set

ABSTRACT

Maximum intensity projection (MIP) is an important visualization method that has been widely used for the diagnosis of enhanced vessels or bones by rotating or zooming MIP images. With the rapid spread of multidetector-row computed tomography (MDCT) scanners, MDCT scans of a patient generate a large data set. However, previous acceleration methods for MIP rendering of such a data set failed to generate MIP images at interactive rates. In this paper, we propose novel culling methods in both object and image space for interactive MIP rendering of large medical data sets. In object space, for the visibility test of a block, we propose the initial occluder resulting from a preceding image to utilize temporal coherence and increase the block culling ratio a lot. In addition, we propose the hole filling method using the mesh generation and rendering to improve the culling performance during the generation of the initial occluder. In image space, we find out that there is a trade-off between the block culling ratio in object space and the culling efficiency in image space. In this paper, we classify the visible blocks into two types by their visibility. And we propose a balanced culling method by applying a different culling algorithm in image space for each type to utilize the trade-off and improve the rendering speed. Experimental results on twenty CT data sets showed that our method achieved 3.85 times speed up in average without any loss of image quality comparing with conventional bricking method. Using our visibility culling method, we achieved interactive GPU-based MIP rendering of large medical data sets.

1. Introduction

Maximum intensity projection (MIP) is a volume rendering method that is used to find the maximum intensity value along the viewing direction. This technique is widely used for the diagnosis of enhanced vessels or bones by rotating or zooming MIP images because it clearly visualizes high-intensity regions [1]. With ever-increasing resolution and availability of multidetector-row computed tomography (MDCT) scanners, MDCT scans of a patient generate a large data set – more than 500 images. Therefore, interactive rendering of MIP images for large medical data sets without the loss of the image quality is required.

Recently, volume rendering [2,3] has been accelerated using graphics processing unit (GPU) to achieve an interactive speed that augments the level of understanding. Specifically, many acceleration techniques for volume rendering on a GPU have been proposed in order to reduce the overload on the fragment shader [4,5]. Recently, a bricking method without culling on a GPU was proposed to accelerate MIP rendering of large medical data sets [6]. Although these acceleration methods [4–6] on graphics hardware significantly reduce rendering overheads, they are not enough for interactive MIP rendering of large medical data sets with more than 500 slices even though the performance of the recent hardware has much improved.

MIP rendering images do not convey depth information. The visibility is not defined by the distance from the viewing point (i.e., depth value), but by the intensity value of a resampling position. By replacing the depth value with the intensity value, the visibility test for MIP can be efficiently performed in the depth test engine for GPU-based rendering [7,8].

The main contribution of this paper is to suggest novel culling methods in both object and image space for interactive GPU-based MIP rendering of large medical data sets without the loss of the image quality. In object space, we subdivide the volume into several blocks and test the visibility for each block. By replacing the

* Corresponding author. Tel.: +82 2 2164 4911; fax: +82 2 2164 4945.
E-mail addresses: kuei@hansung.ac.kr (H. Kye), bongbong@cau.ac.kr (B.-S. Sohn), jjlee@cglab.snu.ac.kr (J. Lee).

0895-6111/$ – see front matter © 2012 Elsevier Ltd. All rights reserved.
http://dx.doi.org/10.1016/j.compmedimag.2012.04.001

depth value with the density value in the fragment shader [9], we can examine the visibility of each block using the *z*-test [8]. Users frequently change the viewing direction by small amount of angles to estimate the depth cue. So, to increase the visibility culling ratio, we exploit the temporal coherency by using the occlusion map, which is created from the preceding image and called as *the initial occluder*. As a result, both the culling ratio and rendering speed are increased. In addition, we propose the hole filling method using the mesh generation and rendering to improve the culling performance during the generation of our initial occluder. In image space, we find out that there is a trade-off between the block culling ratio in object space and the culling efficiency in image space. In this paper, we propose a balanced culling method by classifying the visible blocks into two types: *very dirty* and *less dirty* blocks. We maximize the block culling ratio in object space for very dirty blocks, which has little gains by culling in image space. And we maximize the culling efficiency in image space for less dirty blocks, which has little gains by block culling.

The remainder of this paper is organized as follows. The next section discusses related works. Section 3 describes the proposed method of block culling using the initial occluder. Section 4 describes the proposed method of image space culling based on the visible block classification. Section 5 presents the experimental results, followed by conclusion in Section 6.

2. Related works

Many volume rendering algorithms have been proposed using GPU-based 3D texture mapping hardware [4,10–12]. Even though GPU performance has much improved, the fragment shader is still overloaded because of the increased resolution of volume and image data [13]. Many acceleration methods, such as empty space leaping and early ray termination [4,5,14], have been applied to GPU-based volume rendering by using the programmable shader functionality [9].

The MIP can be implemented by replacing alpha blending with a maximum selection operation in the blending stage. Instead of a maximum selection operation, the depth test can be used to select the maximum value by setting the depth value as the intensity value [7]. Even though the notions of empty space and opaque pixels are unclear for MIP images since there are no transfer functions or depth relations, visibility culling is possible in a GPU-based MIP algorithm by using the depth value (i.e., intensity value) [8].

Since the size of volume data is increased, the volume can be divided into several blocks, each of which can be sequentially rendered. This bricking method reduces the working set and increases the cache efficiency [6,15–17]. Moreover, the visibility test for each block can be applied easily. The pre-calculated maximum value for each block is used to test whether or not the block can be skipped. If all the pixels that are covered by the block have values higher than the maximum value of the block, the block can be skipped [18].

The occlusion map can be updated during the rendering of each block to increase the culling ratio by maintaining an accurate occluder. While the front-to-back or back-to-front rendering process is required for direct volume rendering [19], the rendering order does not affect the accuracy of MIP. Therefore, the MIP performance can be improved by sorting all of the blocks by their maximum or median intensity and by rendering the high intensity blocks before rendering the low intensity blocks [8].

Since there are so many objects in a scene, reducing the visibility testing time for each object is also important. A hierarchical occlusion map [20] has been used for software-based approaches [15]. Recent GPUs have hierarchical *z*-buffer [21] functionality, such as Hierarchical Z from ATI [22]. The occlusion query, which is a function supported by GPUs, counts the number of pixels during rendering so that the visibility test is accelerated by the GPU [23,24].

For direct volume rendering, the position of the first non-transparent sampling points for each ray in the previous view is stored as the C-buffer (coordinate buffer) [25]. Any empty space can be skipped by transforming the data of the C-buffer to the current view. When the saved points from the previous view are transformed into the current view, holes can be induced. Since the overall performance much decreases due to those holes, the holes must be filled. However, hole filling with neighbor values induces a number of artifacts [26]. Moreover, a large splatting kernel cannot be adapted to MIP, since the density variation near a point cannot be estimated.

Opaque pixels are marked in the stencil or depth buffer to skip additional calculations for direct volume rendering [4]. Since the notion of opaqueness is unclear for MIP, the sampled value should be directly compared with the pixel value. If the comparison is performed on the frame buffer, the read–write conflicts often occur because DirectX and OpenGL have no specification for a simultaneous read and write access to texture [13].

3. Block culling using the initial occluder

Our culling method is composed of novel culling methods in both object and image space for interactive GPU-based MIP rendering of large medical data sets without the loss of the image quality, as shown in Fig. 1. In object space, we subdivide the volume into several blocks and test the visibility for each block. To increase the visibility culling ratio, we exploit the temporal coherency by using the occlusion map, which is created from the preceding image and called as the initial occluder. As a result, both the culling ratio and rendering speed are increased. In addition, we propose the hole filling method using the mesh generation and rendering to improve the culling performance during the generation of our initial occluder. In image space, we propose a balanced culling method by classifying the visible blocks into two types: very dirty and less dirty blocks.

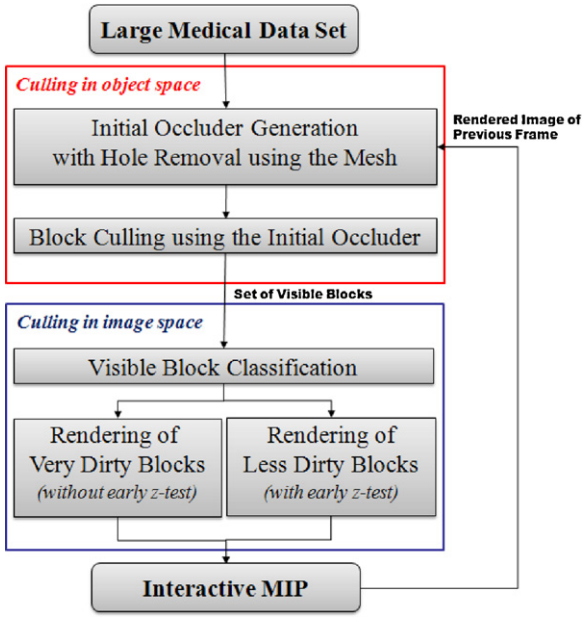

Fig. 1. Process of visibility culling based on the initial occluder and the visible block classification.

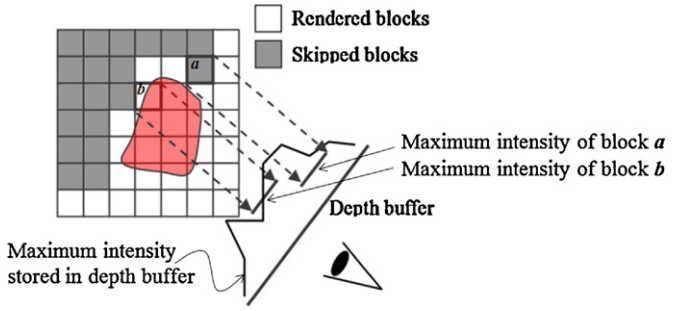

Fig. 2. Block culling using the depth buffer and occlusion query [8].

3.1. Block culling using the occlusion query for MIP rendering

In this section, we briefly explain a previous method to cull blocks in MIP for accelerating the rendering speed [8]. To test the visibility of a mesh in surface rendering, the occlusion query, which is very fast due to the acceleration of modern graphics hardware, can be used. The occlusion query provides the number of pixels that can be updated while the block is rendered. If the occlusion query for a bounding box of some primitives results in a zero, the primitives can be skipped. The occlusion query compares the depth value in order to determine whether each block needs to be rendered or not.

In the previous work [8], we exploit the occlusion query to compare the intensity value since the block does not have to be rendered in MIP rendering when the intensity of every rendered pixels of the block is lower than that of previously rendered pixels. When the bounding box of a block is tested by the occlusion query, the vertices of the bounding box are transformed to the image space. Each z-coordinate of the transformed vertices is replaced with the precalculated maximum intensity value of the block. Then, the depth buffer contains the intensity value rather than the depth value, and the occlusion query compares the intensity value rather than the depth value. So, the block can be culled in MIP rendering if the result of the occlusion query for rendering the bounding box is zero [see a in Fig. 2]. Furthermore, the depth buffer holds the same values of the frame buffer and MIP images can be generated by using the depth test, which selectively updates each pixel, instead of maximum blending, which updates each pixel for every time.

3.2. Initial occluder using temporal coherence

As the objects near the viewer occlude other rear objects in surface rendering, high-intensity objects occlude other low-intensity objects in MIP rendering. In previous works [8,18], the high-intensity blocks were rendered before other low-intensity blocks. In this way, the depth buffer is first filled with the high-intensity blocks so that the culling ratio for the remaining low-intensity blocks becomes higher.

In the diagnosis using MIP, the viewing direction is frequently rotated by small angles because the doctor wants to estimate the depth information from the MIP images having no depth information. Considering that successive MIP images have the strong temporal coherency, we can generate the initial occluder by reprojecting the preceding image to the current view. In this paper, we improve the efficiency of the visibility culling by generating and utilizing this initial occluder for every frame. Before we render each frame, all of the blocks are tested by this initial occluder so that the culling ratio can much be increased. Our initial occluder has three properties as follows:

(1) The generation speed of the initial occluder is fast.

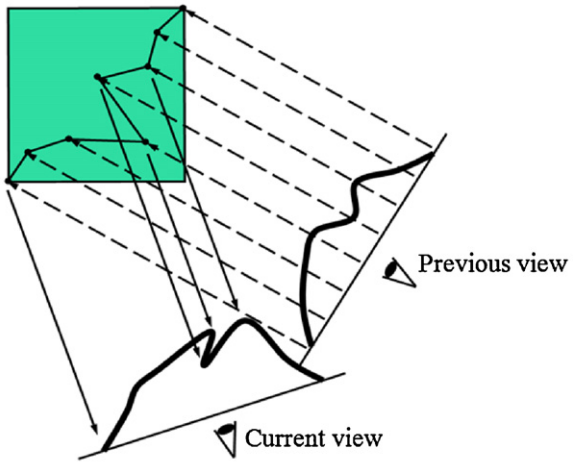

Fig. 3. We store the world coordinate of points having the maximum intensity from the previous view and re-project the points to the current view.

(2) The culling ratio is higher than that of previous sorting algorithm [8].
(3) Visible blocks are not skipped.

In order to re-project the points in preceding coordinates to the current coordinates, the world coordinates of the points in the preceding image are needed. Instead of reconstructing the world coordinates from the pixels in the preceding image, we directly store the world coordinates of the points having the maximum intensity from the previous view [25]. In this way, we can remove the calculation error (see Fig. 3).

We maintain three image buffers. The frame buffer (FB) contains intensity values to display images. The depth buffer (DB) keeps the same intensity values as FB to select the maximum values. The coordinate buffer (CB) contains the world coordinate of each pixel having the maximum intensity in the FB. Assuming an integer time, we have $FB(t-1)$, $DB(t-1)$, and $CB(t-1)$ from the preceding time step $t-1$. The initial occluder using temporal coherence is generated as follows. First, all pixels on $CB(t-1)$ are converted to vertices in world coordinates and vertices are rendered in the current view. As a result, $FB_{init}(t)$, $DB_{init}(t)$, and $CB_{init}(t)$ are generated. We regard the initial occluder as $DB_{init}(t)$ so that all blocks take the visibility test using $DB_{init}(t)$. After the $FB_{init}(t)$, $DB_{init}(t)$, and $CB_{init}(t)$ buffers are updated by rendering all visible blocks, we have $FB(t)$, $DB(t)$, and $CB(t)$.

3.3. Hole removal by the mesh generation and rendering

If we use point splatting in rendering vertices in the current view for the generation of the initial occluder, $FB_{init}(t)$, $DB_{init}(t)$, and $CB_{init}(t)$ will contain many holes. In this paper, we remove holes by connecting the points in $CB(t-1)$ as a mesh. If the resolution of $CB(t-1)$ is $m \times n$ pixels, we create $(m-1)\times(n-1)\times 2$ triangles. We render the mesh instead of splatting points so that the holes are filled with the interpolated values from the points.

However, in the rasterizing stage of rendering the mesh, an interpolation between the sampled values can induce false rendering results (see Fig. 4(a)). The interpolated value ($70=(60+80)/2$) of Fig. 4(a) is not present in the volume data, and it is larger than the real maximum value (50) on the ray. Therefore, this real maximum value (50) is culled, and we obtain the wrong value (70) instead of the correct value (50).

To resolve this problem, we interpolate the world coordinates of the points before texture sampling. Then, we resample the volume (see Fig. 4(b)) using the interpolated coordinates as the texture

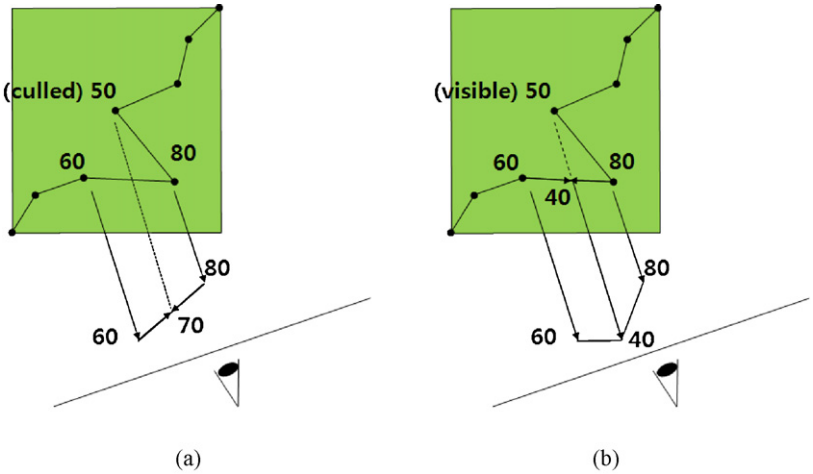

Fig. 4. Comparison between the density interpolation (left) and the texture coordinate interpolation (right). While the density interpolation generates false intensity value and culling, the texture coordinate interpolation generates correct intensity value and culling.

coordinates. In Fig. 4(b), during the mesh rendering (i.e., the initial occluder), we get the interpolated coordinate between the point (60) and the point (80), and then obtain the real value (40) at the interpolated coordinate in the volume data. The rendering results of the initial occluder are 60, 70 and 80 in Fig. 4(a). After the texture coordinate interpolation, they become 60, 40 and 80 in Fig. 4(b). The real maximum value (50) is not culled by the initial occluder value (40) in Fig. 4(b) so that we can get the correct MIP images by hole removal using the mesh generation and rendering.

3.4. Overall process of block culling using the initial occlude with hole filling

The overall process of block culling using the initial occluder with hole filling is explained as follows. The pixels in $CB(t-1)$ are converted to vertices of a mesh. The vertex coordinates are the world coordinates of the pixel, and the texture coordinates are the same as the position. The mesh is rendered using the depth test, and $FB_{init}(t)$, $DB_{init}(t)$, and $CB_{init}(t)$ are generated. This process is very fast because it is simple rendering of a mesh (refer to property 1 of the initial occluder).

Since we use a mesh instead of a point set, holes can be prevented. The visibility test is possible using the occlusion query using $DB_{init}(t)$. While the visible blocks are rendered, $FB_{init}(t)$, $DB_{init}(t)$, and $CB_{init}(t)$ are gradually updated to $FB(t)$, $DB(t)$, and $CB(t)$. If we test visibility and render blocks in a sorted order, the depth values in $DB(t)$ are always greater than those in previous sorting algorithms. Therefore, the culling ratio is increased (refer to property 2 of the initial occluder).

Finally, while the mesh is rendering, the texture coordinates are interpolated, and the volume texture is sampled instead of using an interpolated intensity value. Every depth value of every pixel on $DB_{init}(t)$ is sampled from a position on a ray that passed the pixel. $DB_{init}(t)$ cannot overestimate $DB(t)$, and there is no culling of a visible block (refer to property 3 of the initial occluder).

4. Image space culling of visible blocks using the visible block classification

4.1. Rendering of visible blocks

We use a ray casting method [4,14] rather than the proxy-geometry based method [5,6,8]. Because we have to handle many blocks, the vertex shader can be overloaded by using the proxy-geometry even though current GPUs have a unified pipeline. Moreover, the ray casting method increases the performance gain for culling in image space, which is presented in the next section. For each ray passing a block, we resample the block with a regular step. The intensity values sampled on the texture are compared, and the maximum intensity value and the texture coordinates of the maximum point in the block are extracted. The maximum value is copied to the depth value for the depth test. If the depth value of the ray segment is greater than that of the pixel, the fragment information is updated for the pixel. The intensity, the depth, and the position values are updated into $FB(t)$, $DB(t)$, and $CB(t)$, respectively. After all the blocks are rendered, $FB(t)$ is displayed, and $CB(t)$ is used to generate the initial occluder for time $t+1$.

4.2. Image space culling based on the visible block classification

We have to render a visible block even if just one pixel is updated during the rendering. If the intensity value of a pixel in $DB(t)$ is greater than the maximum value of a block, the ray passing the pixel can be skipped. However, direct reading from $DB(t)$ induces a heavy overhead because of the read–write conflict [13]. We find that the efficient utilization of current GPUs allows us to read and write the depth buffer simultaneously without affecting the overhead.

In this paper, we propose a novel method to use the depth test for image space culling. For each visible block, we replace the depth value with the maximum intensity value of the block. Then, a comparison is made between the intensity value of a pixel in $DB(t)$ and the maximum intensity value of the block in the depth test stage. Therefore, the culling in image space can be directly performed in the graphics pipeline (Algorithm 1). Since the depth test is performed in the last stage of the graphics pipeline and current GPUs have early z-test functionality, a z-test is performed before the fragment shader stage (see Algorithm 1, line 4). A precondition of the early z-test is that the depth value should not be changed in the fragment shader [4] to prevent the wrong result. However, in Algorithm 1, considering that the depth value is replaced with the sample value in the fragment shader (see Algorithm 1, line 7), we have to modify Algorithm 1, which is currently impossible to implement.

Algorithm 1. *An ideal, but impossible image space culling algorithm. The early z-test in line 4 is automatically disabled on the GPU because the depth value is modified in line* 7. *Either line* 4 *or* 7 *has to be removed.*

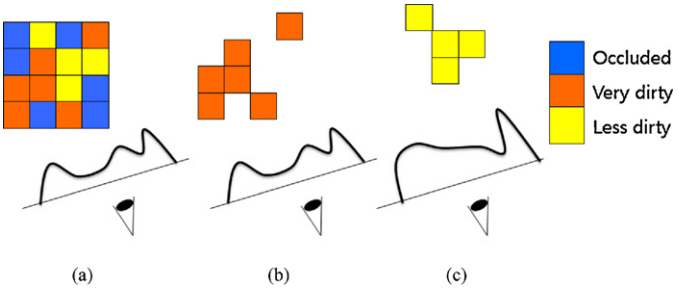

Fig. 5. Rendering sequence for image space culling based on the visible block classification. (a) After the initial occluder is created, the visibility test is applied to the blocks and occluded blocks are skipped from rendering. (b) Very dirty blocks are rendered earlier than less dirty blocks. While the very dirty blocks are rendered, the depth buffer is updated. (c) The less dirty blocks are rendered using the updated depth buffer.

```
1:   For each block
2:     Sample.depth = block.max//from vertex shader
3:     For each pixel
4:       If pixel.depth > sample.depth then exit//early z-test on the GPU
5:         max_value = MAX{sample values along a ray
         segment}//fragment shader start
6:       sample.color = max_value
7:       sample.depth = max_value
8:       sample.position = the position of max_value
9:       If pixel.depth < sample.depth then//depth test on the GPU
10:        pixel.color = sample.color
11:        pixel.depth = sample.depth
12:        pixel.position = sample.position
```

To update $FB(t)$ without modifying the depth value in the fragment shader, we can use a blending operation instead of the depth test. However, this approach causes a problem because the blending operation updates only $FB(t)$, and not $DB(t)$ and $CB(t)$. The blending operation cannot affect the depth value, and blending between the coordinate values is meaningless. After rendering all of the blocks, $FB(t)$ becomes the output image through updating, but $DB(t)$ and $CB(t)$ are not updated from $DB_{init}(t)$ and $CB_{init}(t)$, respectively. Even though the algorithm does not produce incorrect images, the block culling ratio dramatically decreases over time because of the poor estimation of the initial occluder.

We find out that there is a trade-off between the block culling ratio in object space and the culling efficiency in image space. In this paper, to utilize this trade-off for the development a balanced culling method, we classify the visible blocks into two types. The occlusion query provides the number of pixels that can be updated while the block is rendered. We introduce a threshold number T, and, if the result of the occlusion query of a block is larger than T, we classify the block as *very dirty*. If the result is less than T, we classify the block as *less dirty*. And we propose a balanced culling method by applying a different culling algorithm in image space for each type to utilize the trade-off and improve the rendering speed.

The overall procedure of image space culling based on the visible block classification is shown in Fig. 5. After the visibility test of all blocks, we skip all the invisible blocks. Among visible blocks, very dirty blocks are rendered by using Algorithm 2(a) without the early z-test so that $FB(t)$, $DB(t)$, and $CB(t)$ are updated. Less dirty blocks are then rendered using Algorithm 2(b) to benefit from the early z-test so that only $FB(t)$ is updated. After rendering, $FB(t)$ is displayed and $CB(t)$ is used to generate the initial occluder at $t+1$. Therefore, $DB(t)$ and $CB(t)$ are updated only for very dirty blocks.

Algorithm 2. *Rendering algorithms for very dirty blocks (a) and less dirty blocks (b). For very dirty blocks, the early-z test is removed, because the gain from an early-z test is smaller. For less dirty blocks, the updates of depth (DB(t)) and position (CB(t)) are ignored because there are fewer updates.*

```
(a)
1:   For each block
2:     sample.depth = block.max//from vertex shader
3:     For each pixel
         //fragment shader start
4:       max_value = MAX{sample values along a ray segment}
5:       sample.color = max_value
6:       sample.depth = max_value
7:       sample.position = the position of max_value
8:       If pixel.depth < sample.depth then//update using the depth test
9:         pixel.color = sample.color
10:        pixel.depth = sample.depth
11:        pixel.position = sample.position

(b)
1:   For each block
2:     sample.depth = block.max//from vertex shader
3:     For each pixel
4:       if pixel.depth > sample.depth then exit//early z-test on the GPU
         //fragment shader start
5:       max_value = MAX{sample values along a ray segment}
6:       pixel.color = MAX(pixel.color, max_value)//update using a blending
       operation
```

We assume that many pixels are updated while a very dirty block is rendered so that culling in image space and the early z-test are not frequent. We can assume that the gain for the image space culling of the very dirty blocks is small. On the other hand, there are a few pixels to be updated for a less dirty block so that $DB(t)$ and $CB(t)$ do not have to be updated frequently.

When there are too many less dirty blocks at time t, the depth values on the initial occluder are inaccurate and underestimated so that the number of very dirty blocks is increased at time $t+1$. On the other hand, many very dirty blocks at time t result in an accurate initial occluder and, therefore, reduce the number of very dirty blocks at time $t+1$. As time passes, the number of less dirty blocks and very dirty blocks become balanced.

4.3. Overall process of image space culling based on the visible block classification

The overall process of image space culling based on the classification of visible blocks is explained as follows. Using the initial occluder, all blocks take the visibility test. Only very dirty blocks are rendered using Algorithm 2(a), which updates $FB(t)$, $DB(t)$, and $CB(t)$. The less dirty blocks are then rendered using Algorithm 2(b), which uses a blending operation and the early z-test. Since only $FB(t)$ is updated, $FB(t)$ and $DB(t)$ are no longer the same. In this process, the culling ratio of the blocks is slightly decreased because

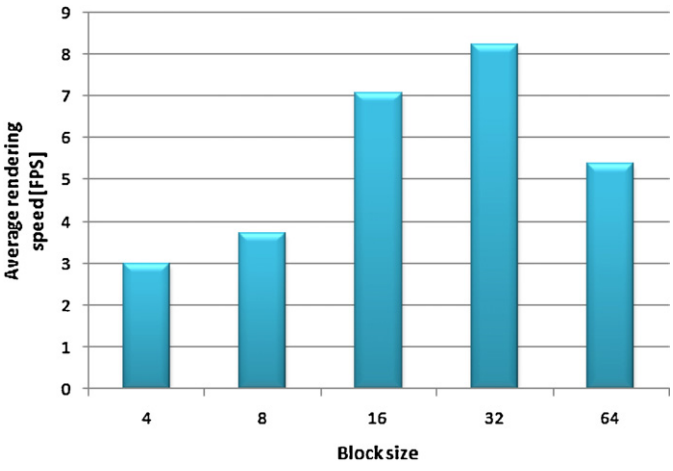

Fig. 6. Average rendering speed with the varying block size.

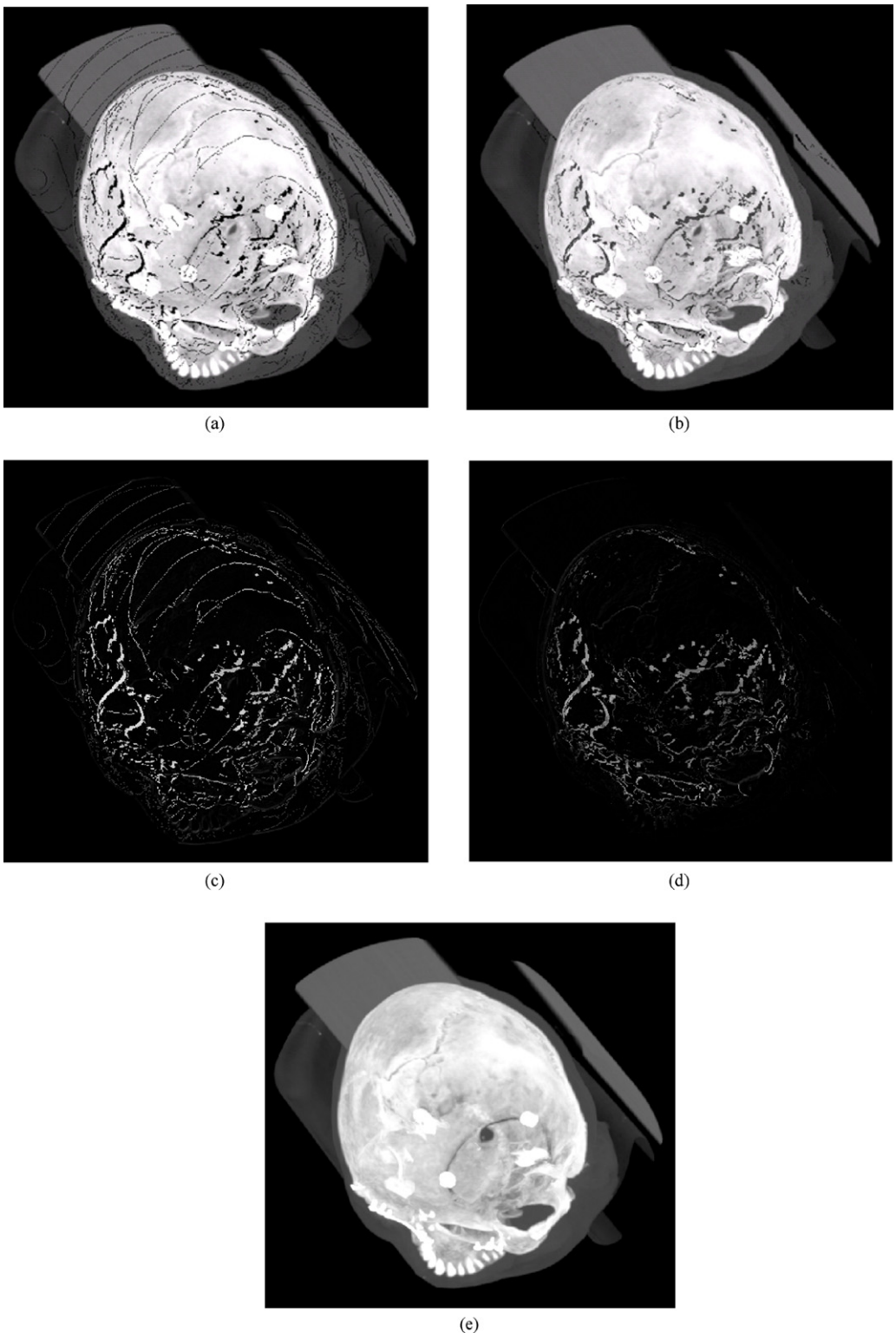

Fig. 7. Comparison of the initial occluder with point splatting and hole removal using temporal coherence. By using point splatting, there are many holes in (a). Due to our hole removal algorithm, every hole is removed in (b). Note that some dark regions in (b) are not holes (black color) but low density regions (gray color) such as the brain, which is the midpoint between the forehead and the occipital. The difference image between the initial occluder with point splatting and the final MIP image rendered by our method in (e) is shown in (c). The difference image between the initial occluder with hole removal and the final MIP image rendered by our method in (e) is shown in (d).

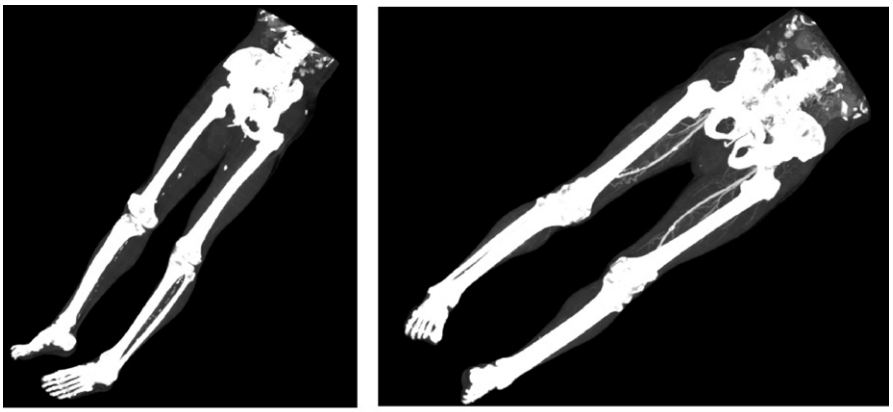

Fig. 8. MIP rendering images of test data sets (a) Angio and (b) legs.

$DB(t)$ is not updated for the less dirty blocks. However, the overall speed is increased due to culling in image space. Finally, we display $FB(t)$ and render the points from pixels in $CB(t)$ to generate $FB_{init}(t+1)$, $DB_{init}(t+1)$, and $CB_{init}(t+1)$.

5. Experimental results

5.1. Experimental environments with parameter settings

The experiments were performed using an Intel i7 desktop system with a 2.66 GHz processor and 8 GB of main memory. The graphics hardware was ATI Radeon 5800 with 1 GB graphics memory. We implemented our method using DirectX [9] and its programmable shader. Instead, GLSL [27] or Cg [28] can be used for the implementation. We tested proposed method on twenty angiography or lower body CT data sets, each of which was from a different patient. The number of images per scan ranged from 581 to 1171. Each image had a matrix size of 512×512. A section thickness of 1.0 mm was used, and pixel sizes ranged from 0.6 to 0.7 mm. Since the size of a pixel on a resulting image is determined to be equal to the size of a voxel, the dimension of the rendering window (i.e., the image resolution) varies from 800×800 to 1200×1200 as the volume resolution changes. To generate a high-quality image, we used the high quality tri-cubic filtering [29] which intepolates neighbor 64 voxels instead of 8 voxels for one resampling. We measured the average rendering speed per scan while each data set was rotated from 0° to 360° with an angle increase of 1° around the diagonal axis (1 1 1).

Considering the cache efficiency, the size of a block should be small because the culling ratio improves with smaller blocks. However, the overhead for handling blocks increases with the number of blocks. From a number of experiments, the optimal block size was determined as $32 \times 32 \times 32$ to maximize the computational efficiency, as shown in Fig. 6.

The possible range of a threshold number T, which classifies the block as very dirty or less dirty, is from 0 to the projected area of a block in the image plane. The optimal block size was determined as $32 \times 32 \times 32$. So, the maximum projected area of a block in the image plane was about 2600 pixels when the block was viewed in the diagonal direction and the projected shape was the hexagon with the edge length of 32 pixels. And the minimum projected area of a block in the image plane was 1024 pixels (i.e., 32×32). The optimal threshold number T was determined as 850 to maximize the computational efficiency.

5.2. Comparison of the generation methods of the initial occluder

We compared the initial occluder using hole removal with the initial occluder using point splatting (Fig. 7). By using point splatting to generate the initial occluder, it contained many holes, as shown in Fig. 7(a). By using proposed hole removal algorithm, the holes were filled with the sampled value, as shown in Fig. 7(b). The difference image between the initial occluder using point splatting and the final MIP image rendered by our method in Fig. 7(e) was shown in Fig. 7(c). The difference image between the initial occluder using hole removal and the final MIP image rendered by our method in Fig. 7(e) is shown in Fig. 7(d). The image of the initial occluder using hole removal in Fig. 7(b) is not the final MIP image, but only the intermediate internal data structure for the acceleration of visibility culling. Using this initial occluder image, the acceleration of visibility culling is performed. If there is more similarity between this intermediate image and final MIP image, the culling performance is improved better.

5.3. Evaluation of the image quality of our MIP rendering

The MIP rendering results of test data sets were shown in Fig. 8. We culled only unnecessary blocks or pixels, so there was no degradation of the image quality. In addition, we evaluate the image quality of our MIP rendering methods by subtracting all the MIP rendering images of proposed method from those of the basic GPU-based MIP method [4] without any optimization while each data set was rotated with from 0° to 360° with an angle increase of 1° around the diagonal axis (1 1 1). All the subtraction images from twenty data sets with 360 viewing directions did not have any non-zero intensity pixel. We concluded that there was no loss of image quality in proposed method.

5.4. Comparison with the conventional method

We compared the computational efficiency of the proposed method with that of conventional bricking method proposed by Kiefer et al. [6]. In addition, we compared the proposed method ($\text{Proposed}_{\text{VBC}}$) using both the initial occluder and visible block classification with the proposed method ($\text{Proposed}_{\text{IO}}$) using only the initial occluder. Table 1 reported the average rendering speed (in frames per second, FPS) for these three algorithms.

By culling in object space with the initial occluder ($\text{Proposed}_{\text{IO}}$), we have improved the average rendering speed 1.68 times faster than conventional bricking method [6]. Using both the initial

Table 1
Performance comparison with conventional method.

	Conventional method [6]	$Proposed_{IO}$	$Proposed_{VBC}$
Average FPS	2.14	3.59	8.24
Speedup with conventional method [6]	1.00	1.68	3.85

Table 2
Comparison of average block culling ratio.

	$Proposed_{IO}$	$Proposed_{VBC}$
Average block culling ratio	45.44%	40.08%

occluder and visible block classification ($Proposed_{VBC}$), we achieved 3.85 times speedup of MIP rendering in average without any loss of image quality comparing with conventional bricking method [6]. Proposed method based on the initial occluder and visible block classification was noticeably faster than conventional method. Moreover, the improvement becomes larger as the size of data set becomes bigger so that our algorithm is suitable for interactive MIP rendering of large medical data sets.

Table 2 presented the average block culling ratio. Depending on the characteristics of the data sets, about 40% of blocks were culled using proposed method ($Proposed_{VBC}$). The average culling ratio of $Proposed_{VBC}$ was slightly less than that of $Proposed_{IO}$, because the less dirty blocks did not update the depth buffer when image space culling was applied. However, the visible blocks were efficiently rendered by culling in image space so that the overall rendering speed of proposed method ($Proposed_{VBC}$) wais the fastest, as shown in Table 1.

6. Conclusion

In this paper, we proposed novel culling methods in both object and image space for interactive MIP rendering of large medical data sets. In object space, we presented a new method to cull more blocks than previous methods. Using temporal coherence, the preceding image is re-projected to a current view to quickly generate an initial occlude, which improved the culling ratio a lot and rendering speed. In addition, we proposed the hole filling method using the mesh generation and rendering to improve the culling performance during the generation of our initial occluder. In image space, we skipped unnecessary pixels by using the visible block classification and exploiting the early *z*-test while the visible blocks are rendered. Since the early *z*-test presumed that the depth values should not be modified in the fragment shader stage, we classified the visible blocks into two types and applied a different rendering algorithm for each type.

As shown in the experimental results, we achieved noticeable performance improvement of rendering speed without the loss of image quality so that interactive MIP rendering of large medical data sets was possible. Our culling method is suitable for an interactive application to produce high-quality volume rendering images, such as for a medical imaging diagnosis system. In future work, multi-resolution data structures such as octree [30,31] can be integrated into our method for the further acceleration of MIP rendering speed. Proposed method can improve the diagnostic efficiency of a doctor by enabling the interactive MIP visualization of large medical data sets (e.g., lower extremity CT angiography).

Conflict of interest

None declared.

Acknowledgements

This Research was financially supported by Hansung University. And this research was supported by Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Education, Science and Technology (No. 2011-0014041).

References

[1] Iserhardt-Bauer S, Hastreiter P, Tomandl B, Köostner N, Schempershofe M, Nissen U, et al. Standardized analysis of intracranial aneurysms using digital video sequences. Proc Med Image Comput Comput Assist Interv 2002:411–8.
[2] Levoy M. Efficient ray tracing of volume data. ACM Trans Graphics 1990;9:245–61.
[3] Lacroute P, Levoy M. Fast volume rendering using a shear–warp factorization of the viewing transformation. In: Proceeding of ACM SIGGRAPH. 1994. p. 451–8.
[4] Kruger J, Westermann R. Acceleration techniques for GPU-based volume rendering. In: Proceedings of IEEE Visual. 2003. p. 287–92.
[5] Li W, Mueller K, Kaufman A. Empty space skipping and occlusion clipping for texture-based volume rendering. In: Proceedings of IEEE Visual. 2003. p. 317–24.
[6] Kiefer G, Lehmann H, Weese J. Fast maximum intensity projections of large medical data sets by exploiting hierarchical memory architectures. IEEE Trans Inf Technol Biomed 2006;10:385–94.
[7] Heidrich W, McCool M, Stevens J. Interactive maximum projection volume rendering. Proc IEEE Visual 1995:11–8.
[8] Kye H, Jeong D. Accelerated MIP based on GPU using block clipping and the occlusion query. Comput Graphics 2008;32:283–92.
[9] Available at: http://www.microsoft.com/directx/.
[10] Rezk-Salama C, Engel K, Bauer M, Greiner G, Ertl T. Interactive volume on standard PC graphics hardware using multi-textures and multi-stage rasterization. In: Proceedings of the ACM SIGGRAPH/EUROGRAPHICS workshop on graphics hardware. 2000. p. 109–18.
[11] Westermann R, Ertl T. Efficiently using graphics hardware in volume rendering applications. Proc ACM SIGGRAPH 1998:169–78.
[12] Engel K, Ertl T. Interactive high-quality volume rendering with flexible consumer graphics hardware. In: Eurographics State-of-the-Art (STAR) Report. 2002. p. 1–24.
[13] Engel K, Hadwiger M, Kniss J, Rezk-Salama C, Weiskopf D. Real-time volume graphics. AK Peters; 2006.
[14] Roettger S, Guthe S, Weiskopf D, Ertl T, Strasser W. Smart hardware-accelerated volume rendering. In: Proceedings of the Eurographics/IEEE TVCG symposium on visualization. 2003. p. 231–8.
[15] Parker S, Parker M, Livnat Y, Sloan PP, Hansen C, Shirley P. Interactive ray tracing for volume visualization. IEEE Trans Visual Comput Graphics 1999;5(3):238–50.
[16] Grimm S, Bruckner S, Kanitsar A, Groller ME. Memory efficient acceleration structures and techniques for CPU-based volume raycasting of large data. In: Proceedings of IEEE/SIGGRAPH symposium on volume visualization and graphics. 2004. p. 1–8.
[17] Hadwiger M, Markus C, Sigg H, Scharsach K, Bühler M, Gross. Real-time raycasting and advanced shading of discrete isosurfaces. Comput Graphics Forum 2005;24(3):303–12.
[18] Mora B, Ebert DS. Low-complexity maximum intensity projection. ACM Trans Graphics 2005;24:1392–416.
[19] Xie K, Yang J, Zhu YM. Real-time visualization of large volume datasets on standard PC hardware. Comput Methods Programs Biomed 2008;90:117–23.
[20] Zhang H, Manocha D, Hudson T, Hoff KE. Visibility culling using hierarchical occlusion maps. In: Proceedings of the 24th Annual Conference on computer graphics and interactive techniques. 1997. p. 77–88.
[21] Greene N, Kass M, Miller G. Hierarchical z-buffer visibility. In: Proceeding of ACM SIGGRAPH. 1993. p. 231–8.
[22] Persson E. Depth in-depth, ATI Technical report; 2007.
[23] Bartz D, MeiXner M, Hü ttner T. OpenGL-assisted occlusion culling for large polygonal models. Comput Graphics 1999;23:667–79.
[24] Bittner J, Wimmer M, Piringer H, Purgathofer W. Coherent hierarchical culling: hardware occlusion queries made useful. In: Proceeding of EUROGRAPHICS. 2004. p. 615–24.
[25] Yagel R, Shi Z. Accelerating volume animation by space-leaping. In: Proceeding of IEEE Visual. 1993. p. 62–9.
[26] Klein T, Strengert M, Stegmaier S, Ertl T. Exploiting frame-to-frame coherence for accelerating high-quality volume raycasting on graphics hardware. In: Proceeding of IEEE Visual. 2005. p. 123–230.
[27] Wolff D. OpenGL 4. 0 shading language cookbook. Packt Publishing; 2011.
[28] Fernando R, Fernando R. The Cg tutorial: the definitive guide to programmable real-time graphics. Addison-Wesley; 2003.
[29] Bjorke K. GPU gems: high-quality filtering. Addison-Wesley; 2004. pp. 391–415.
[30] Pharr M, Fernando R, Sweeney T. GPU gems 2: programming techniques for high-performance graphics and general-purpose computation. Addison-Wesley; 2005.
[31] Zhang H, Manocha D, Hudson T, Hoff K. Visibility culling using hierarchical occlusion maps. In: Proceedings of ACM SIGGRAPH. 1997. p. 77–88.

Heewon Kye is an assistant professor in the Dept. of information systems engineering, Hansung Univertisy, Seoul, Korea. He received his B.S. degree in computer science and M.S. and Ph.D. degree in computer engineering from Seoul National University, Korea. His research interests include volume rendering, real-time graphics, and medical imaging.

Bong-Soo Sohn is an associate professor in the School of Computer Science and Engineering at Chung-Ang University, Seoul, Korea. He received the M.S. and Ph.D. degrees in Computer Sciences from the University of Texas at Austin, in 2001 and 2005 respectively. He received the B.S. degree in computer science from Seoul National University, Seoul, Korea. His research interests are in the areas of graphics, visualization and 3D image processing.

Jeongjin Lee is an assistant professor in the Department of Digital Media and the director of the Biomedical Imaging Laboratory at the Catholic University of Korea. He received the B.S. degree in mechanical engineering, and the M.S. and Ph.D. degrees in computer science and engineering from Seoul National University, Korea in 2008. His research interests are image registration, image segmentation, computer-aided surgery, and virtual endoscopy.